HW8

Nathan Krieger

2026-03-29

Problem 1: Concept Review

(a) (practice)

Explain in plain language (using limited statistics terminology) why lasso can set some of the regression coefficients to be 0 exactly, while ridge regression cannot. You may include a figure if that is helpful.

Ridge shrinks coefficients towards zero while lasso can bring them all the way to zero. This is because of the shape of the constraint functions. Lasso uses a diamond shape while ridge uses a circular shape.

(b)

Suppose we estimate the regression coefficients in a linear regression model by minimizing

for a particular value of lambda. For parts (a) through (e), indicate which of i. through v. is correct. Justify your answer.

a.

As we increase lambda from 0, the training MSE will:

i. Increase initially, and then eventually start decreasing in an inverted U shape.

ii. Decrease initially, and then eventually start increasing in a U shape.

iii. Steadily increase.

iv. Steadily decrease.

v. Remain constant.

iii

b.

Repeat (a) for test MSE.

ii

c.

Repeat (a) for variance.

iv

d.

Repeat (a) for (squared) bias.

iii

e.

Repeat (a) for irreducible error.

v

Problem 2: Regularized Regression

For this problem, we will use the College data set in the ISLR2 R package. Our aim is to predict the number of applications (Apps) received using the other variables in the dataset.

(a)

Split the data set into a training and a test set. Please set.seed(12) so that we can all have the same results.

```
library(ISLR2)

set.seed(12)

x <- model.matrix(Apps ~ ., data = College)
y <- College$Apps

train <- sample(1:nrow(x), nrow(x) / 2)
test <- setdiff(1:nrow(x), train)

x_train <- x[train, ]
x_test <- x[test, ]
y_train <- y[train]
y_test <- y[test]
```

(b)

Fit a ridge regression model (using all predictors) on the training set. The function glmnet, by default, internally scales the predictor variables so that they will have standard deviation

1. Explain why this scaling is necessary when implementing regularized models.

```
library(glmnet)
```

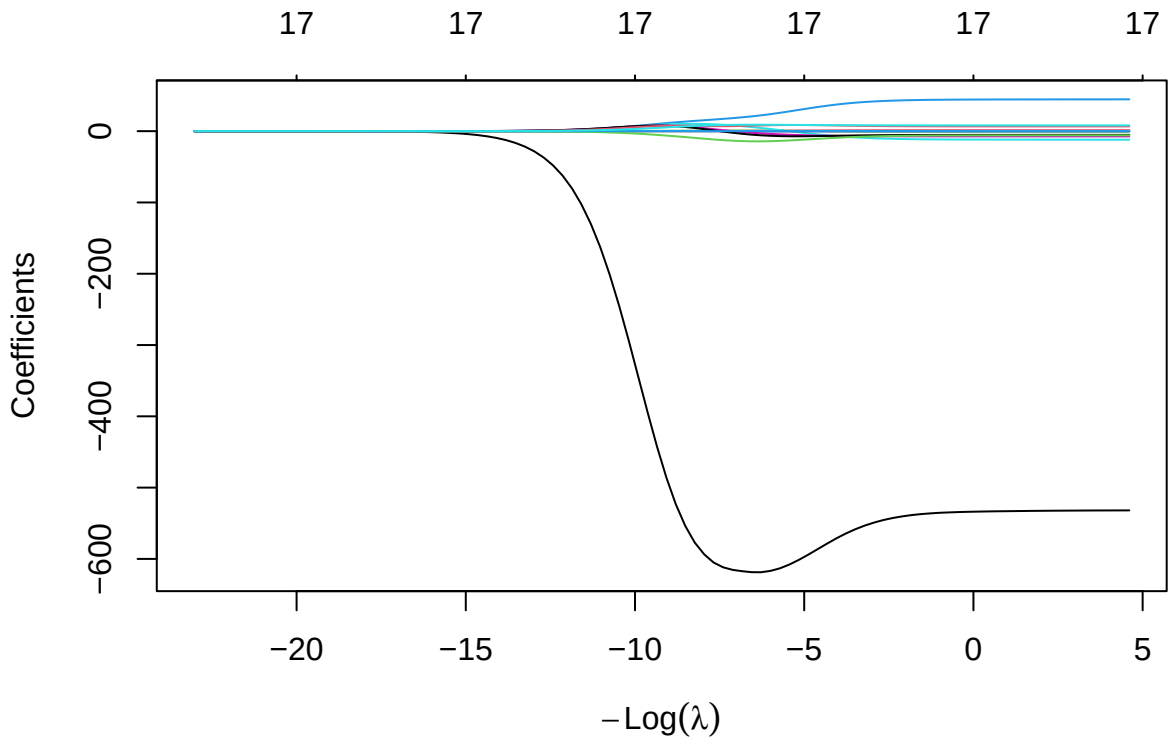
```
## Loading required package: Matrix
```

```
## Loaded glmnet 4.1-10
```

```
grid <- 10^seq(-2, 10, length = 100)

ridge_model <- glmnet(x_train, y_train, alpha = 0, lambda = grid)

plot(ridge_model)
```




```
# ?glmnet
```

Scaling is necessary because the penalty term is applied directly to the magnitude of the coefficients. Sox if one coefficient has a small scale and another has a very large scale, the one with the small scale will have a much larger coefficient to have the same impact on the model and the penalty term would penalize the larger coefficient more. Scaling makes sure all variables are penalized fairly.

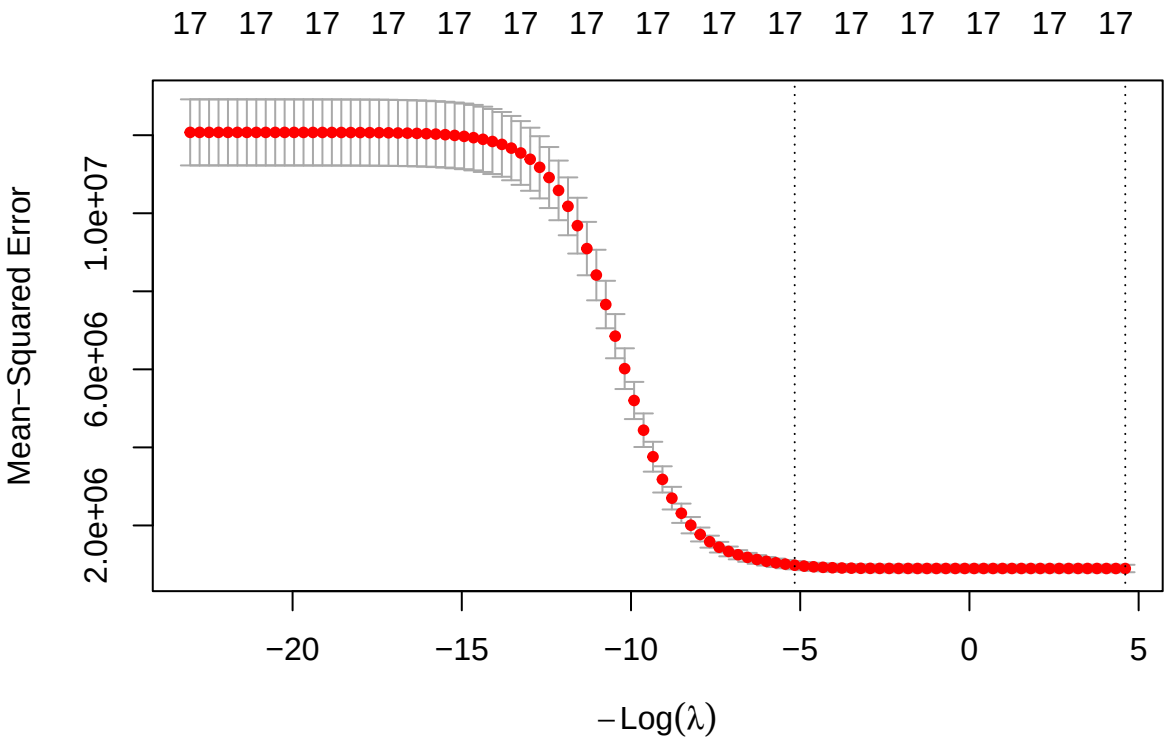
(c)

Find an optimal lambda for the ridge regression model on the training set by using 5-fold cross- validation. Report the optimal lambda here.

```
set.seed(12)

cv_out_ridge <- cv.glmnet(x_train, y_train, alpha = 0, lambda = grid, nfold = 5)

plot(cv_out_ridge)
```




```
optimal_lambda <- cv_out_ridge$lambda.min
optimal_lambda
```

```
## [1] 0.01
```

Optimal Lambda = 0.01

(d) (practice)

What is the value of the l2 norm of the estimated regression coefficients (excluding the intercept) associated with the optimal lambda for ridge regression? You can evaluate this from the training set.

```
ridge_coefs <- as.matrix(coef(cv_out_ridge, s = "lambda.min"))[-1, ]
l2_norm <- sqrt(sum(ridge_coefs^2))

cat("L2 Norm of Ridge coefficients:", l2_norm, "\n")
```

```
## L2 Norm of Ridge coefficients: 534.147
```

(e)

Using that optimal lambda, evaluate your trained ridge regression model on the test set. Report the test MSE obtained.

```
pred <- predict(cv_out_ridge, s = optimal_lambda, newx = x_test)

test_mse = mean((pred - y_test)^2)
test_mse
```

```
## [1] 1416737
```

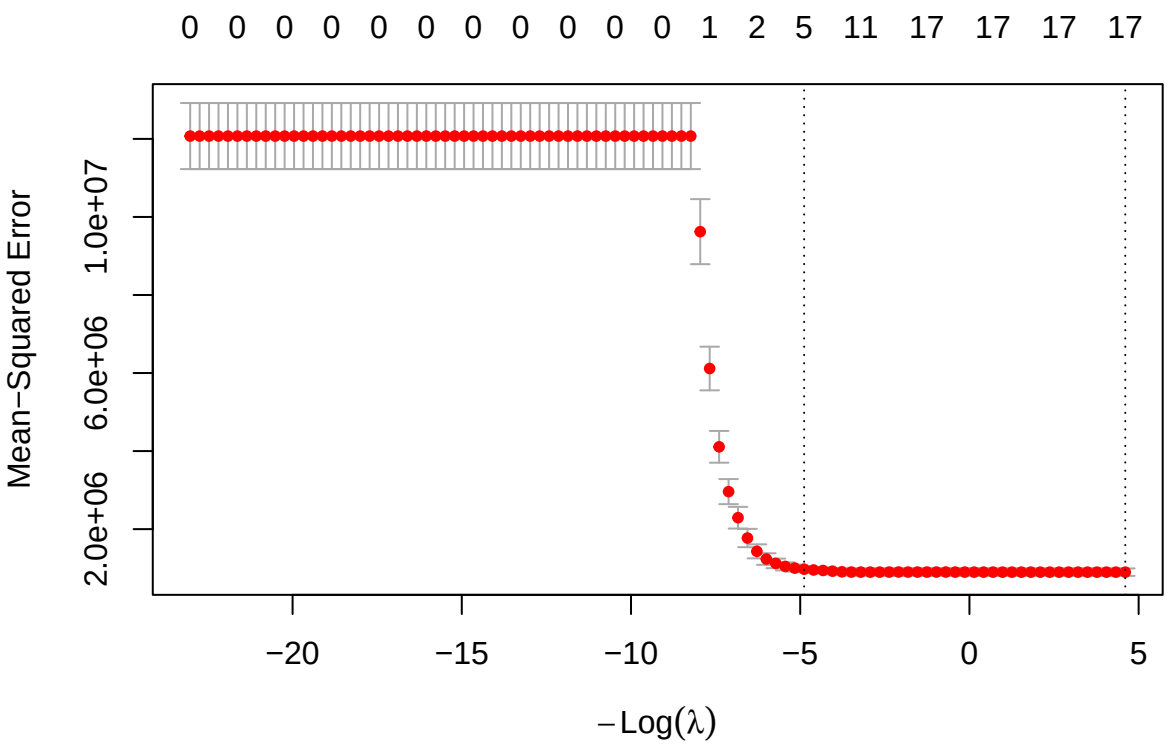
(f)

Find an optimal lambda for the lasso regression model on the training set by using 5-fold cross- validation. Report the optimal lambda here.

```
set.seed(12)

cv_out_lasso <- cv.glmnet(x_train, y_train, alpha = 1, lambda = grid, nfold = 5)

plot(cv_out_lasso)
```




```
optimal_lasso_lambda <- cv_out_lasso$lambda.min
optimal_lasso_lambda
```

```
## [1] 0.01
```

Optimal Lambda = 0.01

(g) (practice)

What is the value of the l1 norm of the estimated regression coefficients (excluding the intercept) associated with the optimal lambda for lasso regression? You can evaluate this from the training set.

```
lasso_coefs <- as.matrix(coef(cv_out_lasso, s = "lambda.min"))[-1, ]
l1_norm <- sum(abs(lasso_coefs))

cat("L1 Norm of Lasso coefficients:", l1_norm, "\n")
```

```
## L1 Norm of Lasso coefficients: 625.6039
```

(h)

Using that optimal lambda, evaluate your trained lasso regression model on the test set. Report the test MSE obtained.

```r
pred <- predict(cv_out_lasso, s = optimal_lasso_lambda, newx = x_test)

test_mse = mean((pred - y_test)^2)
test_mse
```

```
## [1] 1417067
```

(i)

Comment on your results. How accurately can we predict the number of college applications received? Is there much difference among the test errors resulting from these two approaches?

There is not a huge difference in the test errors between the two approaches. In my case, ridge performed slightly better than lasso. The test MSE is 1416737 for ridge. This means that on average the estimates are off by 1190.

Problem 3: Tuning lambda (practice)

For this problem, we will continue with the Hitters example from lecture. Our aim is to predict the salary of baseball players based on their career statistics.

(a)

We will start with a little data cleaning. We will also split the data into a training and test set. So that we all get the same results, please use the following code:

```r
library(ISLR2)
Hitters = na.omit(Hitters)
n = nrow(Hitters) #there are 263 observations
x = model.matrix(Salary ~.,data=Hitters)[,-1] #19 predictors
Y = Hitters$Salary
set.seed(1)
train = sample(1:nrow(x), nrow(x)/2)
test = (-train)
Y.test = Y[test]
```

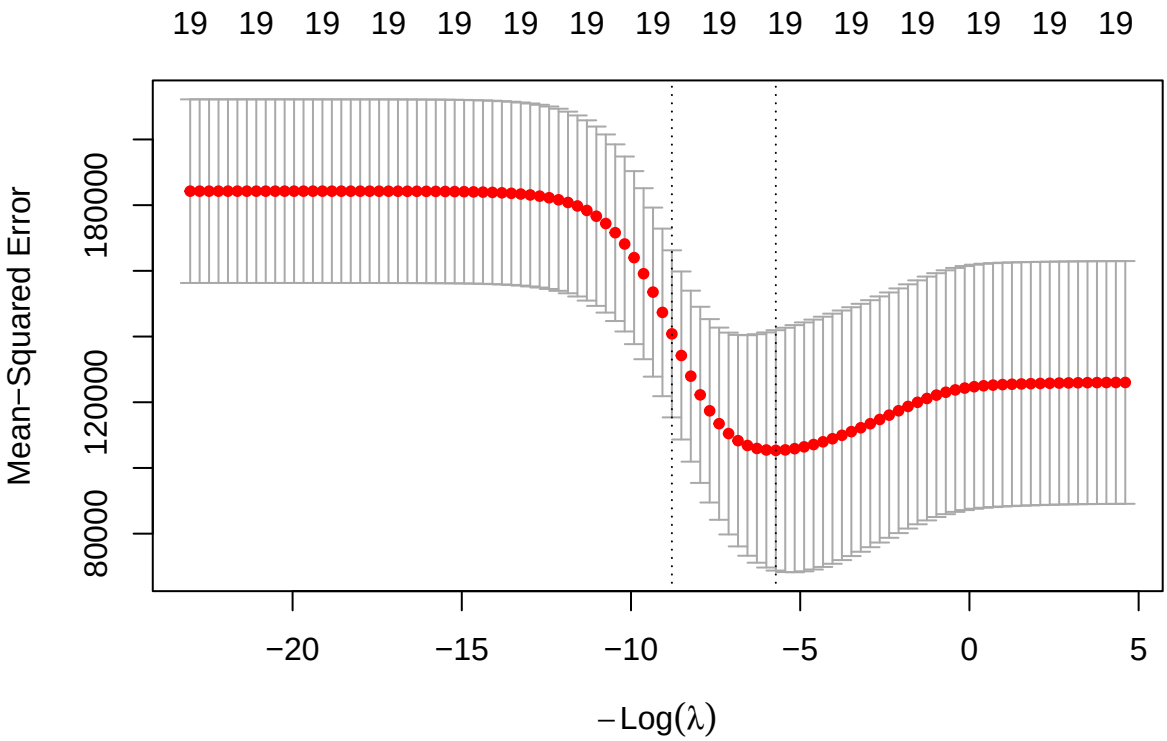
(b)

Fit a ridge regression model. Replicate the example we had in class to obtain the optimal lambda that minimizes the 10-fold CV. Present a plot of the cross-validation error as a function of lambda. Report that value here and call it lambdaridgemin

```r
set.seed(1)

cv_ridge <- cv.glmnet(x[train, ], Y[train], alpha = 0, lambda = grid, nfolds = 10)

plot(cv_ridge)
```




```
lambda_ridge_min <- cv_ridge$lambda.min

lambda_ridge_min
```

```
## [1] 305.3856
```

(c)

Naturally, if we had taken a different training/test set or a different set of folds to carry out cross-validation, our optimal lambda and therefore test error would change. An alternative is to select lambda using the one-standard error rule. The idea is, instead of picking the lambda that produces the smallest CV error, we pick the model whose CV error is within one standard error of the lowest point on the curve you produced in part (b). The intention is to produce a more parsimonious model. The glmnet function does all of this hard work for you and we can extract the lambda based on this rule using the following code: cv.out$lambda.1se (assuming your cv.glmnet object is named cv.out). Report that lambda here and call it lambdaridge 1se .

```
lambda_1se_ridge <- cv_ridge$lambda.1se
lambda_1se_ridge
```

```
## [1] 6579.332
```

6579.332

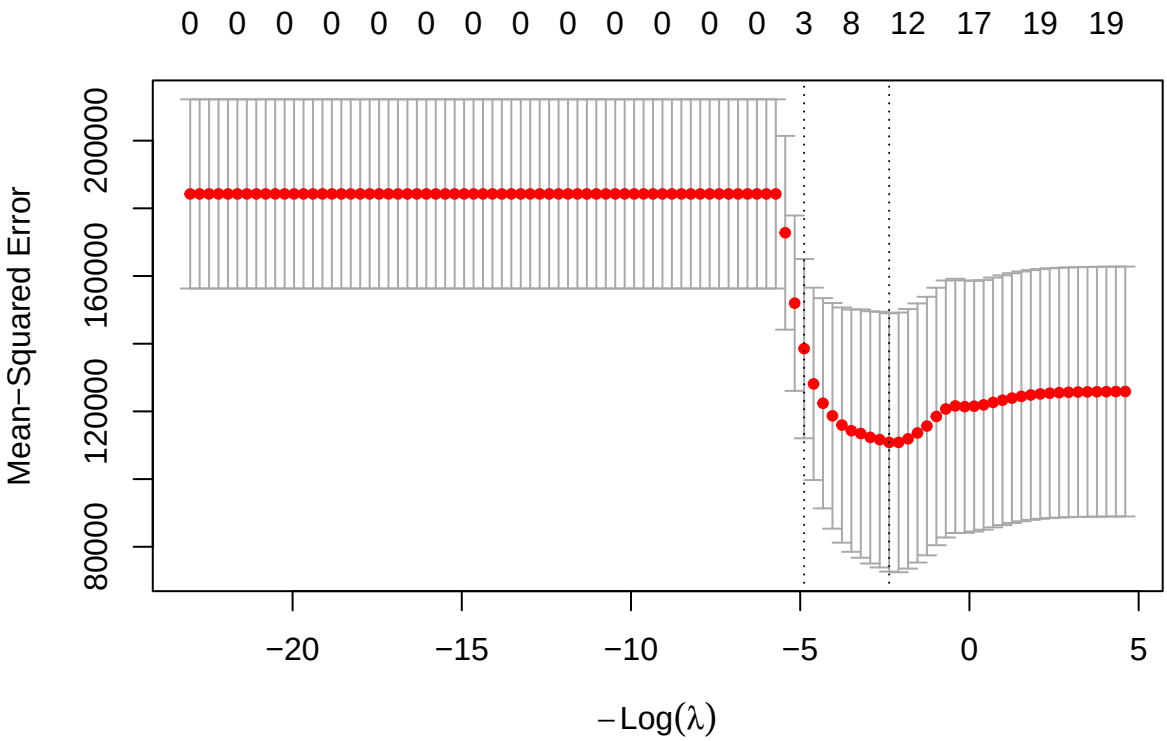
(d)

Fit a lasso regression model. Replicate the example we had in class to obtain the optimal lambda that minimizes the 10-fold CV. Present a plot of the cross-validation error as a function of lambda. Report that value here and call it lambdalasso min .

```
set.seed(1)

cv_lasso <- cv.glmnet(x[train, ], Y[train], alpha = 1, lambda = grid, nfold = 10)

plot(cv_lasso)
```




```
lambdalasso <- cv_lasso$lambda.min
lambdalasso
```

```
## [1] 10.72267
```

10.72267

(e)

For lasso, report the optimal lambda using the smallest standard error rule and call it lambdalasso 1se .

```
lambda_lasso_1se <- cv_lasso$lambda.1se
lambda_lasso_1se
```

```
## [1] 132.1941
```

132.1941

(f)

Evaluate the ridge regression models on your test set using lambda = lambdaridge min and lambda = lambdaridge1se . Evaluate the lasso models on your test set using lambdalassomin and lambdalasso1se . Report the test MSEs from these 4 models.

```
# ridge min
pred_ridge_min <- predict(cv_ridge, s = lambda_ridge_min, newx = x[test, ])

mse_ridge_min <- mean((pred_ridge_min- Y.test)^2)

mse_ridge_min
```

```
## [1] 140081.9
```

```
# ridge 1se
pred_ridge_1se <- predict(cv_ridge, s = lambda_1se_ridge, newx = x[test, ])
```

```r
mse_ridge_1se <- mean((pred_ridge_1se - Y.test)^2)

mse_ridge_1se
```

```
## [1] 165523.3
```

```r
# lasso min
pred_lasso_min <- predict(cv_lasso, s = lambdalasso, newx = x[test, ])

mse_lasso_min <- mean((pred_lasso_min- Y.test)^2)

mse_lasso_min
```

```
## [1] 143527.7
```

```r
# lasso 1se
pred_lasso_1se <- predict(cv_lasso, s = lambda_lasso_1se, newx = x[test, ])

mse_lasso_1se <- mean((pred_lasso_1se- Y.test)^2)

mse_lasso_1se
```

```
## [1] 157773.7
```

(g)

Report the coefficient estimates from ridge using lambdaridge min and lambdaridge1se and likewise for the lasso models. How do the ridge regression estimates compare to those from the lasso? How do the coefficient estimates from using lambdamin compare to those from the one-standard error rule? Discuss what you observe.

```r
set.seed(1)

coef_ridge_min <- predict(cv_ridge, type = "coefficients", s = lambda_ridge_min)
coef_ridge_min
```

```
## 20 x 1 sparse Matrix of class "dgCMatrix"
##               s=305.3856
## (Intercept)  66.55821717
## AtBat         0.03070510
## Hits          0.31718456
## HmRun         2.66888576
## Runs          0.67478450
## RBI           0.72498413
## Walks         1.98220725
## Years        -2.02585376
## CAtBat        0.01293362
## CHits         0.06795173
## CHmRun        0.68800101
## CRuns         0.13079629
## CRBI          0.16521318
## CWalks        0.17662389
## LeagueN      35.90717638
## DivisionW   -89.89152251
## PutOuts       0.10184056
## Assists       0.08703002
## Errors       -0.52085075
```

```
## NewLeagueN   18.62214490
```

```r
coef_ridge_1se <- predict(cv_ridge, type = "coefficients", s = lambda_1se_ridge)
coef_ridge_1se
```

```
## 20 x 1 sparse Matrix of class "dgCMatrix"
##                 s=6579.332
## (Intercept) 353.269801290
## AtBat         0.042018664
## Hits          0.144311795
## HmRun         0.857046957
## Runs          0.292812111
## RBI           0.307977563
## Walks         0.447552095
## Years         1.497362376
## CAtBat        0.005875002
## CHits         0.022404007
## CHmRun        0.183034794
## CRuns         0.044653523
## CRBI          0.047438316
## CWalks        0.057533761
## LeagueN       2.675484213
## DivisionW   -12.696188712
## PutOuts       0.018769711
## Assists       0.004577823
## Errors       -0.010468427
## NewLeagueN    1.275549136
```

```r
coef_lasso_min <- predict(cv_lasso, type = "coefficients", s = lambdalasso)
coef_lasso_min
```

```
## 20 x 1 sparse Matrix of class "dgCMatrix"
##                s=10.72267
## (Intercept)  116.7305083
## AtBat          .
## Hits           .
## HmRun          3.3614540
## Runs           .
## RBI            .
## Walks          4.1129902
## Years         -9.3742690
## CAtBat         .
## CHits          0.1854545
## CHmRun         0.9413517
## CRuns          .
## CRBI           0.3738392
## CWalks         .
## LeagueN       42.9719893
## DivisionW   -132.5835239
## PutOuts        0.1133053
## Assists        0.1357082
## Errors         .
## NewLeagueN     .
```

```r
coef_lasso_1se <- predict(cv_lasso, type = "coefficients", s = lambda_lasso_1se)
coef_lasso_1se
```

```
## 20 x 1 sparse Matrix of class "dgCMatrix"
##               s=132.1941
## (Intercept) 347.0159601
## AtBat         .
## Hits          .
## HmRun         .
## Runs          .
## RBI           .
## Walks         1.2895711
## Years         .
## CAtBat        .
## CHits         .
## CHmRun        0.4889675
## CRuns         .
## CRBI          0.3796004
## CWalks        .
## LeagueN       .
## DivisionW     .
## PutOuts       .
## Assists       .
## Errors        .
## NewLeagueN    .
```

It's clear that the lasso method provides much sparser models. There is also big differences between the min and lse models as shown above.

(h)

Which model performs the best in terms of prediction? Explain any intuition as to why.

Ridge min performs best. Its test MSE was the lowest.

(i)

If you were to make a recommendation to an upcoming baseball player who wants to make it big in the major leagues, what handful of features would you tell this player to focus on?

Walks, CHmRun, and CWalks. These three predictors were present on every optimal model.